Python Lambda Functions: A Reference

Anonymous Functions, Functional Programming Operations & Syntax Workspaces

> Lambda functions in Python are small, anonymous, inline functions defined using the `lambda` keyword. Unlike standard functions declared via `def`, lambdas are restricted to a single expression and automatically return its result without an explicit return statement, making them ideal for lightweight functional patterns.

1. Core Lambda Syntax & Basics

Concept	Instruction & Core Mechanics	Step-by-Step Code & Output

Basic Blueprint	Declared using `lambda arguments: expression`. They take any number of parameters but can only evaluate one inline statement.	`>>> double = lambda x: x * 2`
		`>>> double(7)`
		`14`

Multi-Argument	Comma-separate multiple variables before the colon. The expression evaluates them collectively at run-time.	`>>> add = lambda a, b: a + b`
		`>>> add(10, 5)`
		`15`

2. Functional Operations: Mapping Collections

Function	Instruction & Core Mechanics	Step-by-Step Code & Output

map() with Lambda	Applies the inline lambda logic to every element within a sequence. Wrap the result in `list()` to evaluate the lazy map generator.	`>>> nums = [1, 2, 3, 4]`
		`>>> squares = list(map(lambda x: x**2, nums))`
		`>>> print(squares)`
		`[1, 4, 9, 16]`

3. Functional Operations: Filtering Collections

Function	Instruction & Core Mechanics	Step-by-Step Code & Output

filter() with Lambda	Evaluates each element against a boolean conditional expression. Only items that yield a `True` result are retained in the output collection.	`>>> nums = [1, 2, 3, 4, 5, 6]`
		`>>> evens = list(filter(lambda x: x % 2 == 0, nums))`
		`>>> print(evens)`
		`[2, 4, 6]`

4. Structural Key Sorting Lookups

Lambdas frequently serve as custom weight extraction keys for standard sorting operations across complex datatypes.

Sorting Context	Instruction & Core Mechanics	Step-by-Step Code & Output

Sorting Tuples/Dicts	Pass a lambda to the `key` parameter of `sorted()` or `.sort()` to dictate precisely which coordinate or feature coordinates to sort by.	`>>> pairs = [(1, 'one'), (3, 'three'), (2, 'two')]`
		`>>> sorted_pairs = sorted(pairs, key=lambda item: item[1])`
		`>>> print(sorted_pairs)`
		`[(1, 'one'), (3, 'three'), (2, 'two')]`